

Hands on Activity. <6.1>

<Built in function>

Course Code: CPE007

Program: Computer Engineering

Course Title: Programming logic and design

Date Performed:10/30/2025

Section:CPE11S1

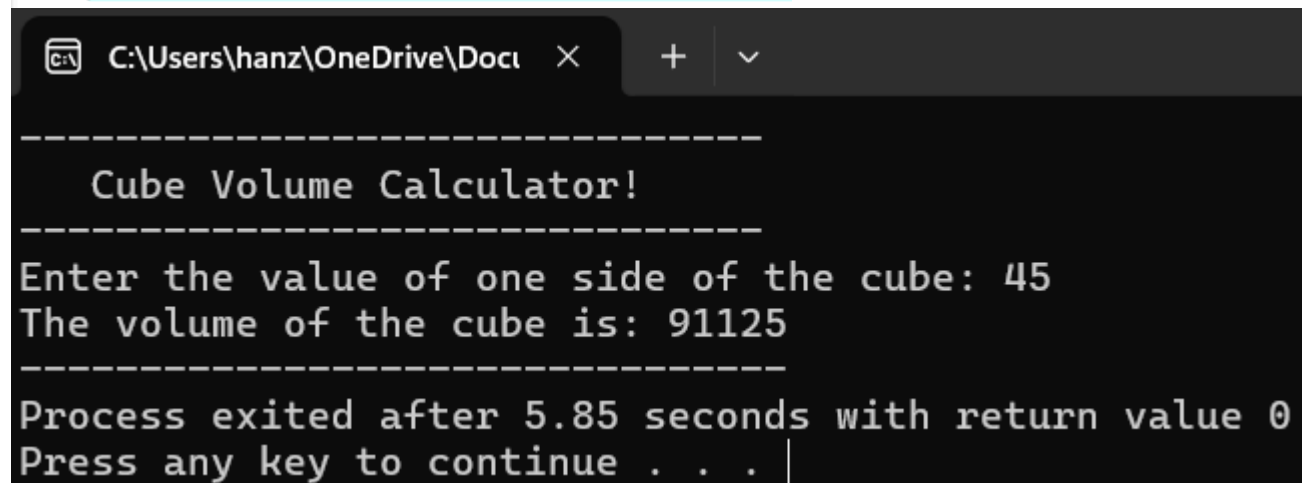
Date Submitted:10/30/2025

Name(s): Hanz Rhayven M. Pabalan

Instructor: Engr. Jimlord Quejado

6. Output

```
[*] built in function 1.cpp
1  #include <iostream>
2  using namespace std;
3
4
5  void greetUser();
6  int volumeComp(int sides);
7
8
9  int main() {
10     greetUser();
11     int s;
12
13     cout << "Enter the value of one side of the cube: ";
14     cin >> s;
15
16     cout << "The volume of the cube is: " << volumeComp(s);
17
18     return 0;
19 }
20 void greetUser () {
21     cout << "-----" << endl;
22     cout << "    Cube Volume Calculator!    " << endl;
23     cout << "-----" << endl;
24 }
25
26 int volumeComp(int sides) {
27     return sides * sides * sides;
28 }
29
```



```
-----
Cube Volume Calculator!
-----
Enter the value of one side of the cube: 45
The volume of the cube is: 91125
-----
Process exited after 5.85 seconds with return value 0
Press any key to continue . . . |
```

ANALYSIS

1. In my program, I started by including `<iostream>` and using namespace `std` so I can use `cout` and `cin` easily. After that, I created a function called `greetUser()` because in this I will put my title or welcome message. Then, I added another function prototype called `volumeComp(int sides)` which will later handle the computation for the volume of the cube. Inside the `main()` function, I declared `int s` to store the value that the user will input. I then called `greetUser()` so that the welcome message prints on the screen when the program starts. After that, I used `cout` to guide the user on what to do, and `cin` to allow the user to enter a number, which will be stored in `s`. Once the input is received, I used `cout` again to display the result of `volumeComp(s)`, which calculates the volume of the

cube. Lastly, the volumeComp function works by multiplying the side value by itself three times ($s \times s \times s$), giving the correct volume of the cube.

2.

built in function 1.cpp [*] built in function 2.cpp

```
1  #include <iostream>
2  #include <cmath>
3  using namespace std;
4
5  double computeHypotenuse(double a, double b);
6
7  void greetUser();
8
9  int main() {
10     double side1, side2;
11
12     greetUser();
13
14     cout << "Enter Side 1 : ";
15     cin >> side1;
16
17     cout << "Enter Side 2 : ";
18     cin >> side2;
19
20     cout << "The Hypotenuse is : " << computeHypotenuse(side1, side2) << endl;
21     cout << "-----" << endl;
22
23     return 0;
24 }
25 void greetUser(){
26     cout << "-----" << endl;
27     cout << "    Hypotenuse Calculator!    " << endl;
28     cout << "-----" << endl;
29 }
30 double computeHypotenuse(double a, double b) {
31     return sqrt(pow(a, 2) + pow(b, 2));
32 }
```



C:\Users\hanz\OneDrive\Docu



Hypotenuse Calculator!

Enter Side 1 : 5

Enter Side 2 : 8

The Hypotenuse is : 9.43398

Process exited after 9.076 seconds with return value 0
Press any key to continue . . . |

ANALYSIS

2. In this program, I included `<cmath>` so I can use the `sqrt()` function for calculating the hypotenuse. I used a void `greetUser()` function to show my title or welcome message. Then, I created a function prototype using `double` because the computation might result in a decimal value. Inside the `main()` function, I declared `double s1` and `s2` to store the user's input. I used `cout` to ask for the lengths and `cin` to get the values from the user. After receiving the inputs, I called the function that computes the hypotenuse using `sqrt(a * a + b * b)`. Lastly, I displayed the result using `cout`. In this program, I created two functions: `celsius()` and `fahrenheit()`. The `celsius()` function converts a temperature from Fahrenheit to Celsius, while the `fahrenheit()` function converts from Celsius to Fahrenheit. I used `double` as the return type because temperature values can have decimals. Inside `main()`, I used two separate loops. The first loop goes from 0 to 100 and prints Celsius values along with their Fahrenheit equivalents by calling the `fahrenheit()` function. The second loop goes from 32 to 212 and prints Fahrenheit values along with their Celsius equivalents by calling the `celsius()` function. I also used formatting so the output looks clean and easy to read in a table form

```
built in function 1.cpp  built in function 2.cpp  built in function 3.cpp
1  #include <iostream>
2  #include <iomanip>
3  using namespace std;
4
5
6  double celsius(double f);
7  double fahrenheit(double c);
8
9  double celsius(double f) {
10     return (f - 32) * 5 / 9;
11 }
12
13 double fahrenheit(double c) {
14     return (c * 9 / 5) + 32;
15 }
16
17 int main() {
18     cout << "-----\n";
19     cout << "Celsius to Fahrenheit    |    Fahrenheit to Celsius\n";
20     cout << "-----\n";
21     cout << setw(10) << "Celsius" << setw(15) << "Fahrenheit"
22     << setw(15) << "Fahrenheit" << setw(15) << "Celsius" << endl;
23     cout << "-----\n";
24
25     int f = 32;
26
27     for (int c = 0; c <= 100; c++) {
28         cout << setw(10) << c
29         << setw(15) << fixed << setprecision(2) << fahrenheit(c)
30         << setw(15) << f
31         << setw(15) << fixed << setprecision(2) << celsius(f)
32         << endl;
33
34         f++;
35         if (f > 212) break;
36     }
37
38     return 0;
39 }
```



C:\Users\hanz\OneDrive\Docu



Celsius to Fahrenheit		Fahrenheit to Celsius	
Celsius	Fahrenheit	Fahrenheit	Celsius
0	32.00	32	0.00
1	33.80	33	0.56
2	35.60	34	1.11
3	37.40	35	1.67
4	39.20	36	2.22
5	41.00	37	2.78
6	42.80	38	3.33
7	44.60	39	3.89
8	46.40	40	4.44
9	48.20	41	5.00
10	50.00	42	5.56
11	51.80	43	6.11
12	53.60	44	6.67
13	55.40	45	7.22
14	57.20	46	7.78
15	59.00	47	8.33
16	60.80	48	8.89
17	62.60	49	9.44
18	64.40	50	10.00
19	66.20	51	10.56
20	68.00	52	11.11
21	69.80	53	11.67
22	71.60	54	12.22
23	73.40	55	12.78
24	75.20	56	13.33
25	77.00	57	13.89
26	78.80	58	14.44
27	80.60	59	15.00
28	82.40	60	15.56
29	84.20	61	16.11
30	86.00	62	16.67
31	87.80	63	17.22
32	89.60	64	17.78
33	91.40	65	18.33
34	93.20	66	18.89

```
C:\Users\hanz\OneDrive\Docu  X + v

66      150.80      98      36.67
67      152.60      99      37.22
68      154.40     100      37.78
69      156.20     101      38.33
70      158.00     102      38.89
71      159.80     103      39.44
72      161.60     104      40.00
73      163.40     105      40.56
74      165.20     106      41.11
75      167.00     107      41.67
76      168.80     108      42.22
77      170.60     109      42.78
78      172.40     110      43.33
79      174.20     111      43.89
80      176.00     112      44.44
81      177.80     113      45.00
82      179.60     114      45.56
83      181.40     115      46.11
84      183.20     116      46.67
85      185.00     117      47.22
86      186.80     118      47.78
87      188.60     119      48.33
88      190.40     120      48.89
89      192.20     121      49.44
90      194.00     122      50.00
91      195.80     123      50.56
92      197.60     124      51.11
93      199.40     125      51.67
94      201.20     126      52.22
95      203.00     127      52.78
96      204.80     128      53.33
97      206.60     129      53.89
98      208.40     130      54.44
99      210.20     131      55.00
100     212.00     132      55.56

-----
Process exited after 0.1704 seconds with return value 0
Press any key to continue . . . |
```

ANALYSIS

3. In this program, I created two functions: `celsius()` and `fahrenheit()`. The `celsius()` function converts a temperature from Fahrenheit to Celsius, while the `fahrenheit()` function converts from Celsius to Fahrenheit. I used `double` as the return type because temperature values can have decimals. Inside `main()`, I used two separate loops. The first loop goes from 0 to 100 and prints Celsius values along with their Fahrenheit equivalents by calling the `fahrenheit()` function. The second loop goes from 32 to 212 and prints Fahrenheit values along with their Celsius equivalents by calling the `celsius()` function. I also used formatting so the output looks clean and easy to read in a table form. Overall, the program is structured so the user can clearly see both conversions without typing anything, and the functions make the code more organized and reusable.

7. Supplementary Activity
8. Conclusion
My conclusion is. I realized how useful functions are in organizing code and making it easier to understand. In each activity, I used different functions whether for displaying a welcome message, calculating the volume of a cube, finding a hypotenuse, or converting temperatures and it made my program cleaner and more structured. By separating the computations into their own functions, I didn't have to repeat code, and the main() function became easier to follow. I also learned the importance of choosing the correct data type, like using double for computations that can produce decimals. Overall, these programs helped me understand how functions improve readability and efficiency, and I can see myself applying this approach to bigger and more complex programs in the future.
9. Assessment Rubric